The learning oracle

A self-contained statement of the framework, as agreed on 2026-08-22

2026-08-22

Contents

1 What this document is **1**

2 The problem the framework solves **2**

3 Notation **2**

4 Recipes and traces **2**

5 The learning oracle, passive form **3**

6 Views **4**

7 The learning oracle, interactive form **5**

8 Parties and the two kinds of game **6**

9 The adversary class **7**

10 Cost, and the ratio between parties **8**

11 The corpus, mapped onto the framework **8**

12 The backdoor game, as far as it is fixed **9**

13 What is deferred, and what remains open about the oracle **10**

1 What this document is

This document states, from nothing, the formal framework that the survey's formalization section will rest on. It assumes the reader knows game-based cryptography and machine learning, and assumes no prior exposure to the framework itself. Every object is defined before it is used. Every claim is labelled as a definition, a fact with a proof, a proposal, or an open question.

Two earlier artifacts exist. The extraction games in `games.tex` are one instance of what is defined here, and Section 11 says where they sit. The survey's frame of four coordinates, access point, criterion, budget and baseline, is recovered in Section 8, so nothing already written is lost, and one coordinate, the budget, is replaced by something more specific.

2 The problem the framework solves

A security definition for a learned model has to say what the adversary may do. For black-box access this is manageable: the adversary sends inputs and receives outputs, and one counts the queries. For white-box access it is not. The adversary holds the weights, and what it can learn from them depends on how training shaped them. A definition that covers that case seems to require a theory of the training map, the function from a corpus to a model. The map itself is public code that anyone can run. What no theory describes is what the map produces, and whether anything it produces can be obtained more cheaply than by running it.

Cryptography faced the same shape of problem with hash functions. No theory describes what SHA-256 does to its input, so the random oracle model replaces the hash function with an oracle that every party queries and no party can simulate, and proves theorems relative to that oracle. The generic group model does the same for group structure, and the Random Language Model of Brzuska and Couteau does it for language membership [4]. In each case the function is public, the hardness lies in reaching a *specified* output without paying the search or the walk, and the cost of an attack is counted in oracle queries.

The framework below makes that move for learning. The one object we cannot describe is the training map, so that is the one object the oracle idealizes. Inference is a computable function, analysis of weights is ordinary computation, and data is supplied by the game. The move has a precedent inside machine learning: the generic learning model of Villa et al. is this idealization used for one theorem, on the tension between a classifier's usefulness and membership privacy [25]. What is new here is using it as the ambient model every game lives in, and as the unit every party is priced in.

3 Notation

$\mathcal{X}, \mathcal{Y}$	input space and output space of a model
D	a corpus, a finite multiset of labelled examples from $\mathcal{X} \times \mathcal{Y}$
n	the security parameter, which scales the corpus, the model and the budgets
R	a recipe, Definition 1
w_t	the state of training after step t, a vector of weights
g_t	the gradient, or other per-step quantity, computed at step t
τ	a training trace, Definition 2
L	the learning oracle, Definitions 3 and 6
V	a view, Definition 4
$\mathcal{C}, \mathcal{A}$	the challenger and the adversary
steps	the cost unit, one step of the recipe

A *party* is any algorithm in a game. The challenger runs the game. The adversary tries to win it. Other named parties, an auditor or a second adversary, appear where a game needs them. Everything is probabilistic polynomial time in n unless a game says otherwise.

4 Recipes and traces

The framework needs a name for a training procedure that is fixed and public but whose behaviour on a given corpus we do not claim to understand.

Definition 1 (Recipe)**.** A recipe R is a public description of a training procedure: an architecture, a loss, an optimizer, a batch rule, an initialization rule, a step count T, and any noise the procedure adds to its own computation. A recipe fixes a *step function* S that carries the current state w_{t-1} and a batch B_t to a per-step quantity g_t and a new state w_t.

A recipe is an object a paper would write in its experimental section. Stochastic gradient descent on a named architecture with a named learning rate is a recipe. Differentially private SGD, which clips and adds Gaussian noise to each gradient, is a different recipe, and the noise is inside it.

Definition 2 (Trace)**.** The trace of running recipe R on corpus D with coins r is the sequence

$$\tau = \big(w_0;\ (B_1, g_1, w_1);\ (B_2, g_2, w_2);\ \ldots;\ (B_T, g_T, w_T)\big),$$

where w_0 is the initial state drawn by the initialization rule, B_t the batch drawn by the batch rule, and $(g_t, w_t) = S(w_{t-1}, B_t)$. The final model is w_T. The *length* of the trace is T.

The trace is everything that happened during training, in order. A paper that releases a model releases w_T. A paper on gradient leakage studies some of the g_t. A proof-of-learning scheme asks a prover to exhibit a trace. These are all views of one object, which is the reason the trace and not the model is the oracle's output.

5 The learning oracle, passive form

Definition 3 (Passive learning oracle)**.** The passive learning oracle L_R for a public recipe R is an oracle that, on input a corpus D, draws fresh coins r, runs R on D with r, and returns the trace τ. It is the only source of traces in any game. Every party that holds a trace, or a model, or any part of a trace, obtained it through a call to L_R made by itself or by another party who then passed a view of it on. No party can compute a trace by any other means.

The last two sentences are the idealization, and they say something different from their random-oracle ancestor. A hash function is public, and so is the recipe. Nobody is prevented from running it. The oracle is a meter, not a keeper of a secret function: it runs the recipe so that the theory need not understand it, and it charges full price. What the idealization asserts is that the meter cannot be cheated, in two ways.

No skipping. The content of a T-step trace costs T steps. A hash chain of length T is computable by anyone, and by assumption not in fewer than T sequential calls; the walk to a converged model is priced the same way. Sequential functions are the fine-grained home of this shape of assumption [17], with one boundary: they price depth, where the step count here is total work. Proof-of-learning, whose security rests on the unproved claim that forging a training record costs as much as training [18], is an instance of this postulate.

No aiming. A hash is inverted only by searching over inputs. Likewise there is no inverse map from desired model properties to weights. A party that wants a model with a specified behaviour, one that agrees with a victim model, contains a backdoor, or forgets a record, chooses corpora and ports and pays more training, because learning is the only search over model space the literature contains.

Both postulates are one clause read from two sides. The honest party's work cannot be compressed, and the adversary's search has no primitive better than the oracle. A party that beats either postulate has, by that act, produced a training shortcut, which is the subject of Proposition 1.

Three consequences follow directly from the definition.

Fact 1 (Inference is free of the oracle)**.** *Given a model w_T, evaluating it on an input, computing its gradient with respect to an input, and any analysis of its weights are ordinary computations the party runs itself. None is an oracle call.*

This is what makes white-box access expressible. The adversary holding w_T may run singular value decomposition on a layer, probe features, or do anything else with the weights. The restriction is on how it *obtains* trained models, not on what it does with them.

Fact 2 (Fresh coins)**.** *Two calls to L_R on the same corpus return two independent traces. A party wanting several models trained on D makes several calls.*

Fact 3 (Cost)**.** *A call on a corpus D with recipe R costs T steps, where T is the recipe's step count, and submits $|D|$ examples. A party's oracle cost is the total number of steps it has caused to be run, and its total submitted corpus size, over all its calls.*

Steps rather than calls are the unit so that the passive oracle and the interactive oracle of Section 7 are charged alike. A shadow model trained from scratch costs T steps. Fine-tuning for T' steps from a held model is a recipe with step count T' whose initialization rule loads that model, and costs T'.

6 Views

A game rarely hands a whole trace to a party. It hands over what the scenario says the party sees, and that is a function of the trace.

Definition 4 (View)**.** A view V is a publicly known function of a trace, possibly randomized and possibly lossy. A game assigns each party a view. When a trace τ is produced, the party receives $\mathsf{V}(\tau)$ and nothing else of τ.

Views are game parameters, not oracle features. The oracle always produces the full trace. This is deliberate: if the oracle returned noisy gradients in one game and summary statistics in another it would be a different oracle per scenario, and there would be no single object to prove things about.

Example 1 (Views in use)**.**

- **Final model.** $\mathsf{V}(\tau) = w_T$. The released-weights setting.
- **Black-box access.** $\mathsf{V}(\tau)$ is a stateful wrapper that answers evaluation queries on w_T and counts them. The party never holds w_T. This is the API setting, and the count is the query budget.
- **Gradients without initialization.** $\mathsf{V}(\tau) = (g_1, \ldots, g_T)$. The party sees every gradient and not w_0.
- **Noisy channel.** $\mathsf{V}(\tau) = (g_t + \xi_t)_{t \le T}$ for independent noise ξ_t drawn by the view.
- **Sampled steps.** $\mathsf{V}(\tau) = (g_t)_{t \in I}$ for a random subset I of steps drawn by the view.
- **Summaries.** $\mathsf{V}(\tau)$ is the per-step mean and variance of the gradients.
- **Nothing.** $\mathsf{V}(\tau) = \perp$. The party learns that a trace was produced.

Fact 4 (Gradients plus initialization determine the model)**.** *For a deterministic step function, w_T is a function of w_0 and $(B_t, g_t)_{t \le T}$. Hence a party whose view contains the initial state and every per-step quantity holds the final model, and a scenario in which a party "sees the gradients but not the model" is only meaningful when its view withholds something.*

Proof. $w_t = S_2(w_{t-1}, B_t)$ for each t, where S_2 is the state coordinate of the step function, and the composition from w_0 is determined. □

Fact 4 is why the view is a named object. It forces a scenario to say what is withheld rather than to say informally that gradients were observed.

Recipe noise against view noise. Differentially private SGD adds noise inside the recipe. The learner trains on noisy gradients and the final model reflects it. A noisy channel adds noise inside a view. The learner trains on clean gradients and only the observer's copy is noisy. These are different objects. A theorem about one says nothing about the other, and a scenario must say which it means.

7 The learning oracle, interactive form

The passive oracle runs a whole training in one call with every part of the procedure honest. Federated learning, split learning and any setting where a party takes part in training while it runs are not of that shape. The party sends something into training at each round and receives something back, and it may deviate from what the recipe expects of it. The interactive oracle exposes the recipe's step as the unit of interaction.

Definition 5 (Ports)**.** A recipe may name *ports*. A port is a position in the step function where a quantity enters from outside or leaves to outside. Each port has an honest rule, the computation the recipe performs there when no external party is attached, and a *validity predicate*, the set of values the recipe accepts at that port.

Example 2 (Ports in two settings)**.** In federated averaging, each client is a port. Its outgoing quantity is the current global model, its incoming quantity is an update, its honest rule is local training on the client's data, and its validity predicate is whatever the server checks, for instance a norm bound. In split learning, the cut between the client half and the server half is a port in each direction. The client sends activations at the cut and receives gradients at the cut. The honest rule on each side is the forward or backward pass of that side.

Definition 6 (Interactive learning oracle)**.** The interactive learning oracle $\mathsf{L}^{\circ}_{\mathsf{R}}$ for a public recipe R with ports holds a training state. A game *attaches* parties to ports before training starts. The oracle then runs one step per call. In each step it sends to every attached party the outgoing quantities of that party's ports, waits for the incoming quantities, applies the validity predicates, substitutes the honest rule at any port that is unattached or whose incoming value is invalid, and advances the state. The trace records the honest and the received values at every port. As in Definition 3, the oracle is the only source of traces.

Attaching a party to a port gives it two things at once. It *reads* the port's outgoing traffic, which is a view of the trace restricted to that port. And it *writes* the port's incoming traffic, which is an intervention in the trace. A party that should only observe is attached to a port with an honest rule it cannot override, which is a read without a write, and is the same as giving it the corresponding view of a passive trace.

Fact 5 (The passive oracle is a special case)**.** *Running* $\mathsf{L}_{\mathsf{R}}^{\circ}$ *for* T *steps with no party attached produces the same distribution over traces as one call to* L_{R}*, and costs the same* T *steps.*

Definition 7 (Write budget)**.** A party attached to a port has a write budget: the number of steps at which it may send a value that differs from the honest rule, and a bound on how far the value may differ, stated in the port's own units. A write budget of zero is read-only attachment.

Write budgets are the second kind of budget in the framework. The first kind, from Fact 3, is what a party spends on the oracle, a read budget. The second is what a party is permitted to change, a write budget. A poisoning fraction is a write budget at the corpus. A malicious federated client's permitted deviation is a write budget at its port. The prior systematization of this area separates adversaries by whether they can read or also write [21]. Here that separation is two numbers.

8 Parties and the two kinds of game

With the oracle in place, a game is a protocol among parties that all share the same oracle. The shape that recurs across the corpus is the following.

Definition 8 (Game skeleton)**.** A game names the following.

1. A recipe R and a distribution over corpora, from which the challenger draws.
2. For each party, its attachment to ports and its write budget (Definitions 5 and 7), and its view of any trace the challenger produces (Definition 4).
3. For each party, a read budget in steps and in submitted examples (Fact 3).
4. An optional *secret*, a value drawn with min-entropy at least h, given to some parties and withheld from others.
5. A *criterion*, of one of the two kinds below.
6. A *baseline*, the value of the criterion achieved without access, or without the secret, or by the best strategy that ignores the trace entirely.

The survey's frame had four coordinates: access point, criterion, budget, and baseline. Items 1 and 2 are the access point, now as attachments and views. Item 5 is the criterion. Items 3 and 4 split the old budget into a read budget and a write budget, and add the secret, which the old frame lacked and which Section 12 needs. Item 6 is the baseline.

Definition 9 (Distinguish criterion)**.** The challenger prepares two *worlds*, two settings of the corpus or of the ports that differ in a stated way. Either a party submits the two and the challenger flips a coin b to pick one, or the challenger draws both. The challenger runs the oracle in world b, hands each party its view, and a designated party outputs a guess b'. Its advantage is $|\Pr[b' = b] - \frac{1}{2}|$.

Definition 10 (Produce criterion)**.** The challenger runs the oracle once, hands each party its view, and a designated party outputs a value. The party wins if the value satisfies a stated predicate within a stated tolerance. Its advantage is its winning probability minus the baseline.

The two criteria are the search and decision forms that cryptography has always had. Membership inference, differential privacy and undetectability are distinguish games. Reconstruction, extraction and forgery are produce games. A backdoor game is both at once, and Section 12 composes them.

Fact 6 (Worlds must differ)**.** *In a distinguish game, if the two worlds induce the same distribution on the designated party's view, its advantage is zero for every strategy, and indistinguishability holds for every bound on every party. A distinguish game is therefore only informative when the game states how the worlds differ and the statement is checkable.*

Proof. The party's output is a function of its view and its coins, and both have the same distribution in either world. □

Fact 6 is the analogue of requiring the two challenge messages in an indistinguishability game to be distinct. It matters for the backdoor game, and Section 12 returns to it.

9 The adversary class

Definition 11 (Learning-oracle algorithm)**.** A learning-oracle algorithm is an algorithm that obtains traces, models and training quantities only through the learning oracle, and otherwise computes freely. Its cost is its read budget, Fact 3, plus its own running time.

This is the adversary class the framework prices. The name in the rest of the document is *generic*, by analogy with generic group algorithms, which use the group only through its operations.

Definition 12 (Structural attack)**.** An attack is structural if it is not a learning-oracle algorithm, that is, if at some point it produces a model, a gradient or a trace without a corresponding oracle call.

The distinction is relative to what the oracle exposes, and two cautions fix its meaning.

First, an attack that uses only oracle calls but exploits a property of the models the recipe produces is generic. Carlini, Jagielski and Mironov extract the parameters of a ReLU network from real-valued outputs alone, by reading the network's piecewise linearity off second finite differences of those outputs [8], and Canales-Martínez et al. make the extraction polynomial in depth [5]. Both use nothing but evaluations of the released model. Both are generic attacks on a structured model class, in the way a generic group algorithm may exploit that the group is cyclic. A generic-model lower bound for extracting ReLU networks must therefore sit at or below their query counts, and the model is calibrated by them rather than refuted.

Second, what the adversary may exploit is fixed by what the game *declares* about the recipe's output. A game that declares the recipe outputs ReLU networks of depth h admits the attacks above. A game that declares nothing beyond "a function" admits less and proves more, for fewer real systems. The declaration is the one place where knowledge of learning enters the framework. The oracle does not remove that knowledge. It confines it to one line of the game.

Proposition 1 (A structural attack is a training shortcut)**.** *Let a game be secure against every generic adversary within a stated budget, and let a structural attack break it within the same budget. Then the attack contains a procedure that obtains the content of a trace, or a model with a specified behaviour, at a cost below the steps the oracle would charge for it. That procedure violates the no-skipping or the no-aiming postulate for the declared recipe class, and is therefore a way to train, or to search model space, faster than the recipe.*

Proof. By Definition 12 the attack produces a trace, model or training quantity without a corresponding oracle call, hence at a cost not accounted in steps. If that cost were at least the steps of the corresponding oracle call, the attack could be rewritten as a generic one within the same budget, contradicting security against generic adversaries. So the cost is lower, which is the definition of a shortcut. □

This is the form in which the win-win theorem of Degwekar, Nakkiran and Vaikuntanathan enters [11]: either the construction is secure against generic adversaries, or a training shortcut exists, and a training shortcut is a learning-theoretic result of independent value. The proposition is close to a restatement of the definitions. Its use is to fix what a generic-model lower bound promises: not that no attack exists, but that any attack beating the bound trains, or searches model space, faster than the recipe, which the literature at present does not know how to do for the recipes this framework is intended for.

10 Cost, and the ratio between parties

Each party's cost has three parts: steps caused, examples submitted, and running time. A security statement bounds the adversary's advantage for every adversary whose cost lies under a stated threshold. The statements the survey cares about most set the threshold relative to an honest party's cost. Fine-grained cryptography writes that relation as an exponent [4, 19]: the adversary is permitted up to the honest cost raised to $1+\delta$, and the question is the largest δ at which security holds. In this framework the honest cost is a step count, which is dimensionless, so raising it to a power is well defined. The three cautions in `games.tex`, Section 6, on the three incompatible conventions for δ and on the ceiling of Barak and Mahmoody being a query bound [3], apply unchanged.

One consequence is specific to the oracle. Every bound proved here is a bound on oracle steps, not on time. Barak and Mahmoody's adversary against Merkle puzzles makes few oracle queries and runs efficiently only in a relativized world where P = NP [3]. The same will hold of any adversary exhibited by a lower-bound argument in this model. A theorem here should say "steps" where it means steps.

11 The corpus, mapped onto the framework

This section places the families the survey covers into Definition 8, to show that nothing in the framework is particular to one of them.

Membership inference. Distinguish. The worlds differ by one record in the corpus. The adversary's view is the final model, or black-box access. The baseline is the blind attack, which Das et al. show beats published attacks on eight benchmarks [10]. A shadow-model attack [22, 7] is generic and costs $N \cdot T$ steps for N shadow models. The verifier of Zhang et al., who cannot sample the null hypothesis because doing so "would require training new large models" [26], is an auditor with a read budget of zero steps.

Membership inference from gradients. Distinguish, with the adversary's view set to gradients, Example 1. Fact 4 says which lossy view the scenario must name.

Gradient inversion. Produce. The view is gradients or activations at a port, and the criterion is reconstruction within a tolerance. The reconstruction-robustness definition of Balle, Cherubin and Hayes already supplies the criterion and the oblivious baseline [1], and the gradient-inversion literature, for instance [2], supplies attacks with no stated criterion. Passive observation of a federated round is a read-only attachment. A malicious server is a write.

Computational differential privacy. Distinguish, with the worlds drawn by the challenger as neighbouring corpora and the distinguisher bounded [20]. Recipe noise, Section 6, is where the mechanism lives.

Evasion. Produce. The criterion is a label change inside a ball around a given input. The baseline is random noise in the same ball, which the concentration results show already succeeds on some tasks [15]. Access is a view: black-box is the metered wrapper, white-box is the final model, and transfer is a shadow model plus the adversary's own gradient computation. The number of gradient steps an attack is allowed, which the evaluation guidance of Carlini et al. declines to bound [6] and which Suya et al. use to rank attacks [24], is the adversary's running time in Definition 11.

Extraction. Produce. The games of `games.tex` are this case with the view set to black-box access, and their two-party pricing is Section 10.

Poisoning. A write budget at the corpus. Availability poisoning is produce, with the criterion an accuracy drop. Certified defences bound the drop for every write within the budget [23]. Robust estimation in high dimensions is the case where the defender is polynomial time and the writer is unbounded except in the fraction it may change [12].

Proof-of-learning. Produce. The criterion is a trace that a verifier accepts. The honest prover's trace comes from T oracle steps, and forging is producing an accepted trace in fewer. Jia et al. require that forging cost at least as much as honest training and conjecture rather than prove it [18]. In this framework the requirement is a bound on the forger's steps relative to T, and the attacks that break their scheme [13] are generic or structural according to whether they call the oracle.

Watermarking. Distinguish, with a secret as the hinge: the detector holds the key and the distinguisher does not, as in Christ, Gunn and Zamir [9].

Filtering and auditing. Distinguish, where the designated party is an auditor whose read budget is stated relative to the model's own cost.

12 The backdoor game, as far as it is fixed

This is the one game that composes both criteria. It is stated here to the extent it has been agreed, with the open decisions marked.

Fixed. Three roles. A *planter* intervenes during training through a write budget at a port or at the corpus. An *auditor* observes through a view and a read budget and outputs a world guess, the distinguish criterion. An *activator* holds a secret drawn with min-entropy h, chosen by the planter, and tries to elicit a stated behaviour from the final model, the produce criterion. The activator's baseline is the success of an activator without the secret, which is the non-replicability property of Goldwasser, Kim, Vaikuntanathan and Zamir [16]. The auditor never holds the secret, because an auditor given the secret detects by running the activator, so undetectability is meaningful only against auditors without it.

Fixed. The planter is generic, Definition 11. The constructions in the corpus that achieve undetectability all give the planter control of training itself and all use cryptographic structure inside the model, digital signatures, continuous LWE, sparse PCA, obfuscation, or a hash circuit compiled into a transformer [16]. A planter that only writes to the corpus and faces an honest recipe is the case those constructions do not cover.

Fixed. The auditor performs detection, not reconstruction. The question is whether some intervention occurred, and the auditor is not asked to say which.

Open 1 (Non-triviality). By Fact 6, a planter whose writes leave the distribution of the auditor's view unchanged is undetectable and has done nothing. The game needs a condition on world 1 that rules this out. The activator's success is one such condition. If the game is to be stated without requiring the backdoor to work at inference, another condition must replace it, for instance a minimum write budget together with a declared difference in the label distribution of the written examples. Which condition to use is not decided.

Open 2 (Where the planter writes). The planter may write at the corpus, at a training port, or by replacing the data distribution itself. These give different games, and the last puts no fraction in the game at all. Which of them the survey's backdoor section treats first is not decided.

Open 3 (A first theorem). For a planter writing an ε-fraction of the corpus and an auditor viewing the corpus with a bounded number of statistical queries, undetectability is a statement that the poisoned corpus is a planted distribution in the query-hard regime. The lower-bound technology for that exists [14]. Whether it transfers to an auditor viewing the final model, rather than the corpus, is not known.

13 What is deferred, and what remains open about the oracle

Deferred 1 (The criterion for generation). The produce criterion assumes a predicate the challenger can evaluate. For a language model asked to be harmless, the predicate is not something we can write down. Villa et al. idealize that predicate as a ground truth and bound the gap to a computable judge [25]. This framework does not yet include that second idealization, and a game about generation needs it.

Deferred 2 (Classification). The tolerance in a produce criterion assumes a metric on outputs. For classifiers under the zero-one metric, the propositions in `games.tex` that rest on a Lipschitz constant do not apply, and the produce games need a separate treatment.

Open 4 (Declaration). Section 9 fixes that what the adversary may exploit is what the game declares about the recipe's output. The framework does not yet say what a declaration may contain. Too little, and every theorem is about random functions. Too much, and the declaration is the theory of learning the oracle was introduced to avoid. The declaration must also assert that the recipe has no known shortcut, because the postulates are false where one exists: a convex recipe such as linear regression has a closed-form endpoint, so the content of its trace is obtainable with none of its steps. The idealization is only credible for recipe classes in the no-known-shortcut regime, as the random oracle model is only credible for hash functions with no known structure. Where that line sits is the main open question about the oracle itself.

Open 5 (Uninstantiability). Random-oracle proofs are known to have instantiations for which the proved theorem fails. Whether the analogous phenomenon arises when the learning oracle is replaced by a concrete recipe is not known, and the cryptographic reference for the random-oracle case is not yet in the corpus and is owed.

References

[1] Borja Balle, Giovanni Cherubin, and Jamie Hayes. Reconstructing training data with informed adversaries. In *IEEE S&P*, 2022.

[2] Mislav Balunović, Dimitar I. Dimitrov, Nikola Jovanović, and Martin Vechev. Lamp: Extracting text from gradients with language model priors. In *Conference on Neural Information Processing Systems*, 2022.

[3] Boaz Barak and Mohammad Mahmoody-Ghidary. Merkle puzzles are optimal — an $o(n^2)$-query attack on any key exchange from a random oracle. In *CRYPTO — stated in the PDF's own footer: "S. Halevi*, 2009.

[4] Chris Brzuska and Geoffroy Couteau. On building fine-grained one-way functions from strong average-case hardness. In *EUROCRYPT*, 2022.

[5] Isaac A. Canales-Martínez, Jorge Chávez-Saab, Anna Hambitzer, Francisco Rodríguez-Henríquez, Nitin Satpute, and Adi Shamir. Polynomial time cryptanalytic extraction of neural network models. In *EUROCRYPT*, 2024.

[6] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Mądry, and Alexey Kurakin. On evaluating adversarial robustness. arXiv:1902.06705, 2019.

[7] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. Membership inference attacks from first principles. In *IEEE S&P*, 2022.

[8] Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. Cryptanalytic extraction of neural network models. In *CRYPTO*, 2020.

[9] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. arXiv:2306.09194, 2023.

[10] Debeshee Das, Jie Zhang, and Florian Tramèr. Blind baselines beat membership inference attacks for foundation models. In *DATA-FM workshop @ ICLR*, 2025.

[11] Akshay Degwekar, Preetum Nakkiran, and Vinod Vaikuntanathan. Computational limitations in robust classification and win-win results. In *COLT*, 2019.

[12] Ilias Diakonikolas, Gautam Kamath, Daniel M. Kane, Jerry Li, Ankur Moitra, and Alistair Stewart. Robust estimators in high dimensions without the computational intractability. arXiv:1604.06443, 2016.

[13] Congyu Fang, Hengrui Jia, Anvith Thudi, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning is currently more broken than you think. arXiv:2208.03567, 2023.

[14] Vitaly Feldman, Elena Grigorescu, Lev Reyzin, Santosh S. Vempala, and Ying Xiao. Statistical algorithms and a lower bound for detecting planted cliques. In *JACM*, 2017.

[15] Justin Gilmer, Luke Metz, Fartash Faghri, Samuel S. Schoenholz, Maithra Raghu, Martin Wattenberg, and Ian Goodfellow. The relationship between high-dimensional geometry and adversarial examples (adversarial spheres). arXiv:1801.02774, 2018.

[16] Shafi Goldwasser, Michael P. Kim, Vinod Vaikuntanathan, and Or Zamir. Planting undetectable backdoors in machine learning models. arXiv:2204.06974, 2022.

[17] Jiaxin Guan and Hart Montgomery. On sequential functions and fine-grained cryptography. In *CRYPTO*, 2024.

[18] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning: Definitions and practice. In *IEEE S&P*, 2021.

[19] Rio LaVigne, Andrea Lincoln, and Virginia Vassilevska Williams. Public-key cryptography in the fine-grained setting. In *CRYPTO*, 2019.

[20] Ilya Mironov, Omkant Pandey, Omer Reingold, and Salil Vadhan. Computational differential privacy. In *CRYPTO — not stated in this PDF; attribution taken from the reference lists of DP20_SoKDifferentialPrivacies*, 2009.

[21] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P. Wellman. Sok: Security and privacy in machine learning. In *IEEE EuroS&P*, 2018.

[22] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. arXiv:1610.05820, 2017.

[23] Jacob Steinhardt, Pang Wei Koh, and Percy Liang. Certified defenses for data poisoning attacks. arXiv:1706.03691, 2017.

[24] Fnu Suya, Anshuman Suri, Tingwei Zhang, Jingtao Hong, Yuan Tian, and David Evans. Sok: Pitfalls in evaluating black-box attacks. arXiv:2310.17534, 2024.

[25] Federico Villa, F. Betül Durak, Tadayoshi Kohno, Tapdig Maharramli, and Franziska Roesner. Extending the formalism and theoretical foundations of cryptography to ai. arXiv:2603.02590, 2026.

[26] Jie Zhang, Debeshee Das, Gautam Kamath, and Florian Tramèr. Position: Membership inference attacks cannot prove that a model was trained on your data. arXiv:2409.19798, 2025.